Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: Shravan P M
Roll No.: 24110026
Registration No.: 24011101101

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

`https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones`

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128\times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32\times 128\times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9.
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

$$70\% \text{ training}, \qquad 15\% \text{ validation}, \qquad 15\% \text{ testing}.$$

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

```
Subset shape: (2400, 128, 9)  class counts: [400 400 400 400 400 400]

Input tensor shape:
Training   : (1680, 128, 9)
Validation : (360,  128, 9)
Testing    : (360,  128, 9)

Number of classes: 6
```

```
Number of features per time step: 9
Sequence length: 128

Class distribution (train): [280 280 280 280 280 280]
Class distribution (val)  : [60 60 60 60 60 60]
Class distribution (test) : [60 60 60 60 60 60]
```

A balanced laboratory subset of $N = 2400$ windows (400 per class) was drawn from the full UCI HAR training partition (7352 windows) and split 70/15/15 into training, validation and testing sets, giving $N_{\text{train}} = 1680$, $N_{\text{val}} = 360$ and $N_{\text{test}} = 360$, with the class balance preserved exactly in every split.

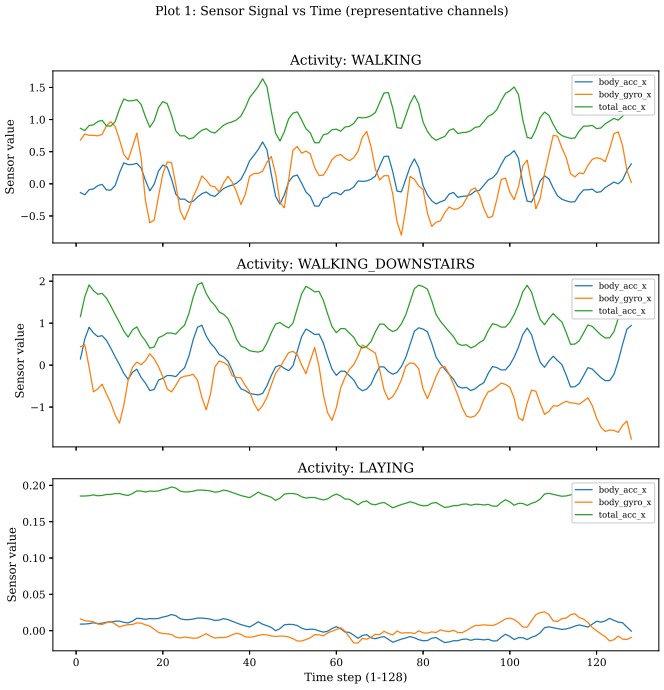
6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

Students should observe that temporal patterns differ across activities.



Required Inference

1. **What temporal pattern is visible?**
   The sensor signals show periodic fluctuations over time, with distinct peaks and troughs during movement. Stationary activities show relatively stable signals with fewer variations.
2. **Which activities exhibit similar patterns?**
   Walking and walking downstairs show similar oscillatory patterns due to continuous body movement. Laying shows a comparatively stable pattern with minimal fluctuations.
3. **Why is the ordering of measurements important?**
   The order preserves the temporal relationship between consecutive sensor readings. This helps identify movement patterns, transitions, and activity-specific changes in the signals.

7. Vanilla RNN Architecture

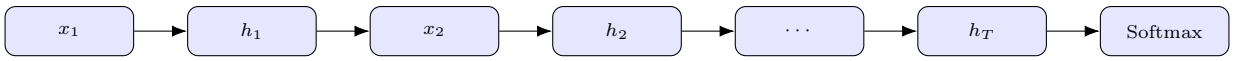
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

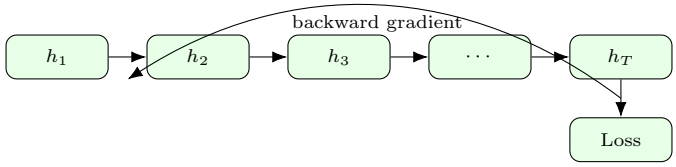
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \to 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \qquad x_2 = 0.7, \qquad x_3 = 0.2,$$

$$h_0 = 0, \qquad W_x = 0.5, \qquad W_h = 0.8, \qquad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1, h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

Program output:

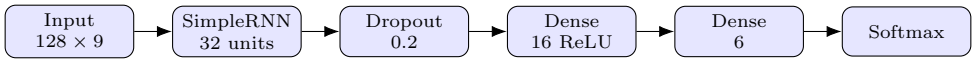
```
h1 = tanh(0.5*0.5 + 0.8*0.336376 + 0.1) = 0.336376
h2 = tanh(0.5*0.7 + 0.8*0.336376 + 0.1) = 0.616352
h3 = tanh(0.5*0.2 + 0.8*0.616352 + 0.1) = 0.599958

h1=0.336376, h2=0.616352, h3=0.599958
```

The manually computed values match the program output to numerical precision, confirming correct application of the recurrence $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.
The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

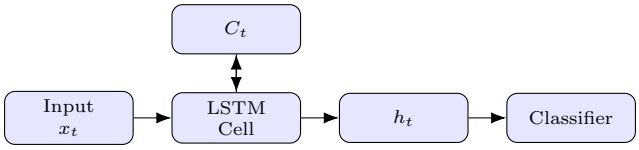
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

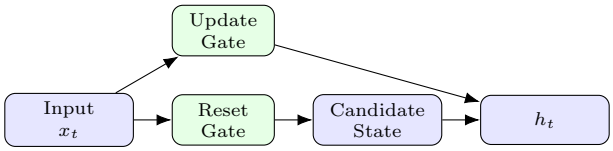
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

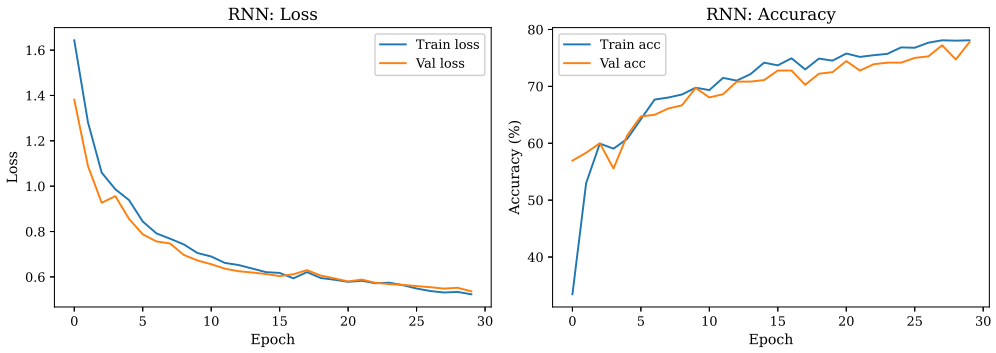
For each of RNN, LSTM and GRU, generate:

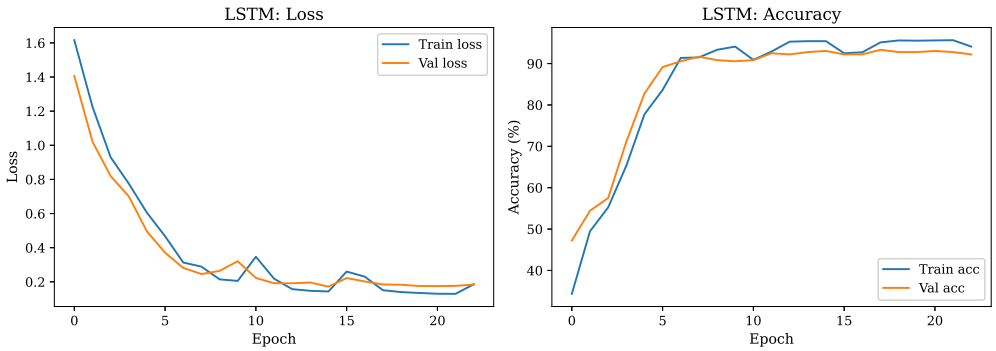
Plot 2: Training and Validation Loss

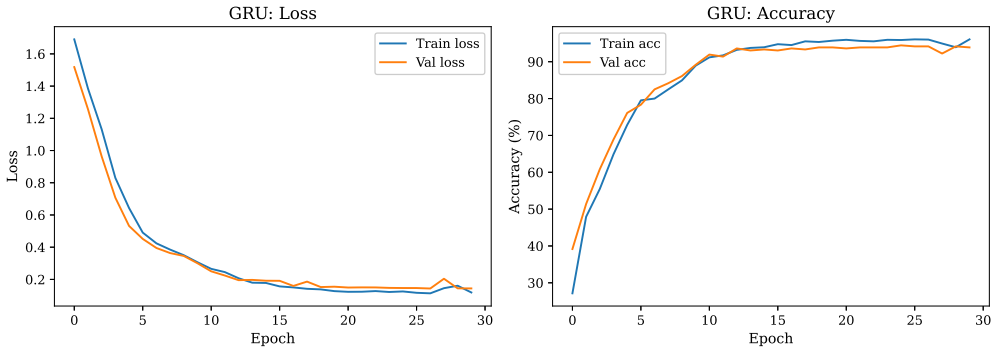
- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy







Inference

RNN

The training and validation losses decrease steadily throughout the epochs, indicating that the model is learning effectively. Both losses remain close to each other, suggesting that there is no significant overfitting. The training and validation accuracies improve gradually and reach approximately 77–78% by the end of training. A small gap between training and validation accuracy is visible, indicating a limited generalization gap. Overall, the RNN converges steadily but achieves lower accuracy compared to LSTM and GRU.

LSTM

The training and validation losses decrease rapidly during the initial epochs and stabilize at low values, indicating good convergence. The training and validation accuracies increase rapidly and reach above 90% within the first few epochs. After convergence, the two accuracy curves remain relatively close, with only a small generalization gap. There are minor fluctuations in the training loss and accuracy, but they do not indicate significant overfitting. Overall, the LSTM demonstrates strong learning performance and good generalization.

GRU

The training and validation losses decrease rapidly and remain very close throughout most of the training process. Both losses stabilize at a low value, indicating that the model converges effectively. Training and validation accuracies increase rapidly and reach approximately 93–95% in the later epochs. The small difference between the two accuracy curves indicates a very low generalization gap and minimal overfitting. Overall, the GRU provides the most stable convergence and achieves the highest overall accuracy among the three models.

Overall Inference

All three models successfully converge, as shown by the decreasing loss and increasing accuracy over the epochs. The RNN achieves the lowest accuracy and shows a slightly larger generalization gap compared with LSTM and GRU. LSTM and GRU converge faster and achieve substantially higher validation accuracy, indicating better learning of temporal dependencies. GRU shows the closest training and validation curves, suggesting particularly good generalization with minimal overfitting. Overall, GRU and LSTM outperform the basic RNN, with GRU providing the best balance of convergence, accuracy, and generalization.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

“‘latex

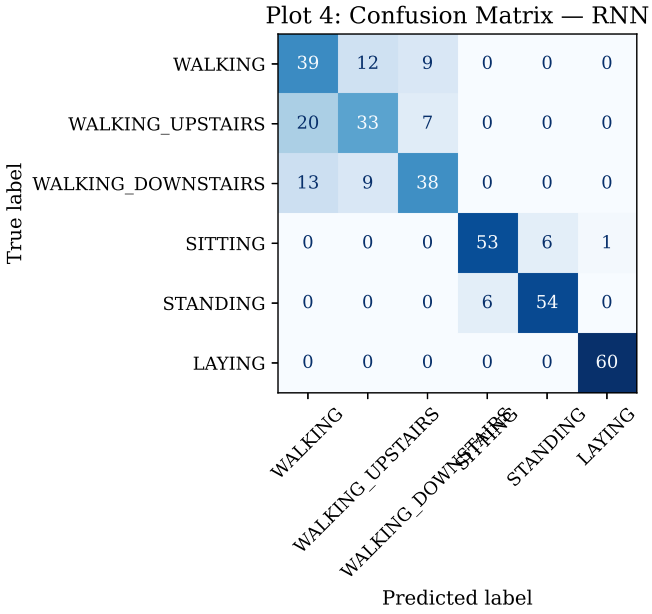
Metric	RNN	LSTM	GRU
Accuracy (%)	76.94	94.72	95.83
Macro Precision (%)	77.31	94.89	95.85
Macro Recall (%)	76.94	94.72	95.83
Macro F1 (%)	76.98	94.71	95.83
Parameters	1974	6006	4758
Training Time (s)	27.6	19.3	20.1

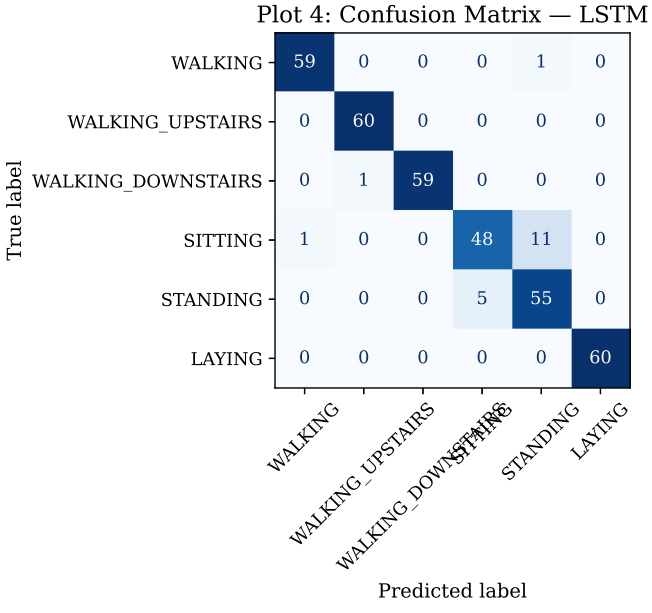
All three models were trained using identical input and output layers, the Adam optimizer with a learning rate of 10^{-3}, a batch size of 32, and the same early-stopping protocol (patience = 8 based on validation loss, with a maximum of 30 epochs). The models were evaluated using the same 1680/360/360 train/validation/test split. “‘

15. Confusion Matrix Analysis

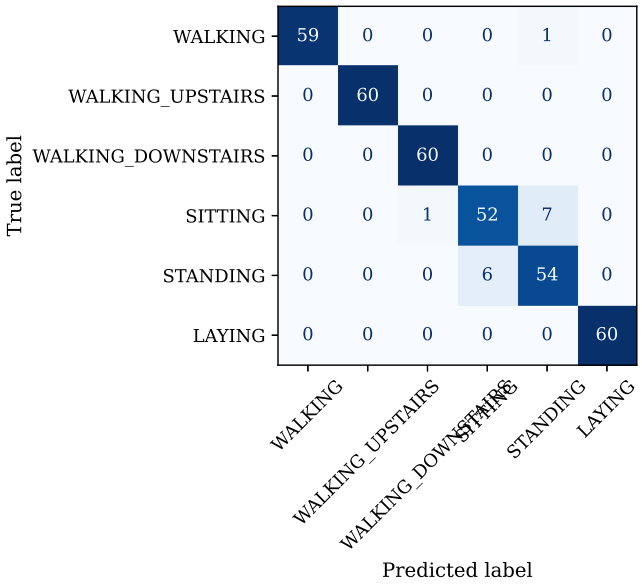
Plot 4: Confusion Matrix

Generate a separate confusion matrix for each model.





Plot 4: Confusion Matrix — GRU



Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

The inference must be based on the student's observed matrix rather than on an assumed result.

Inference from Confusion Matrices

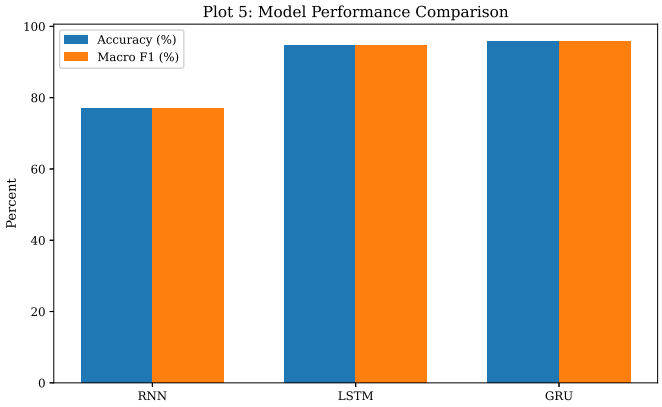
1. **Activity with the highest recognition rate:**
   For the RNN, *LAYING* has the highest recognition rate, with all 60 samples correctly classified. For the LSTM, *WALKING_UPSTAIRS* and *LAYING* are perfectly recognized. For the GRU, *WALKING_UPSTAIRS*, *WALKING_DOWNSTAIRS*, and *LAYING* achieve 100% recognition.
2. **Activity with the largest number of misclassifications:**
   In the RNN, *WALKING_UPSTAIRS* has the largest number of errors, with 27 out of 60 samples misclassified. In both the LSTM and GRU, *SITTING* has the largest number of misclassifications, with 12 and 8 errors respectively.
3. **Frequently confused pairs of activities:**
   The RNN shows considerable confusion among the walking activities, particularly *WALKING–WALKING_UPSTAIRS* and *WALKING–WALKING_DOWNSTAIRS*. For both LSTM and GRU, the most prominent confusion is between *SITTING* and *STANDING*.
4. **Consistency of errors across RNN, LSTM and GRU:**
   The errors are not completely consistent across all three models. The RNN mainly struggles to distinguish between the different walking activities, whereas LSTM and GRU largely resolve these errors. However, *SITTING–STANDING* remains a common source of confusion across all three models, although the number of errors is substantially reduced in LSTM and GRU.

16. RNN vs. LSTM vs. GRU Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time (s)	27.6	19.3	20.1
Test F1-score (%)	76.98	94.71	95.83

Plot 5: Model Performance Comparison

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.



Required inference:

Students should discuss the trade-off among:

$$\text{Predictive Performance} \quad \leftrightarrow \quad \text{Model Complexity} \quad \leftrightarrow \quad \text{Training Cost.}$$

The RNN has the lowest model complexity with only 1,974 parameters, but it also provides the lowest predictive performance with a test F1-score of 76.98%. The LSTM achieves a much higher F1-score of 94.71%, but has the highest complexity with 6,006 parameters. Interestingly, it also has the lowest training time of 19.3 seconds in this experiment. The GRU provides the best predictive performance, achieving a test F1-score of 95.83%, while using 4,758 parameters and requiring 20.1 seconds of training time. Thus, the GRU offers the best overall trade-off between predictive performance, model complexity, and training cost, achieving the highest F1-score without the parameter overhead of the LSTM.

17. Effect of Sequence Length

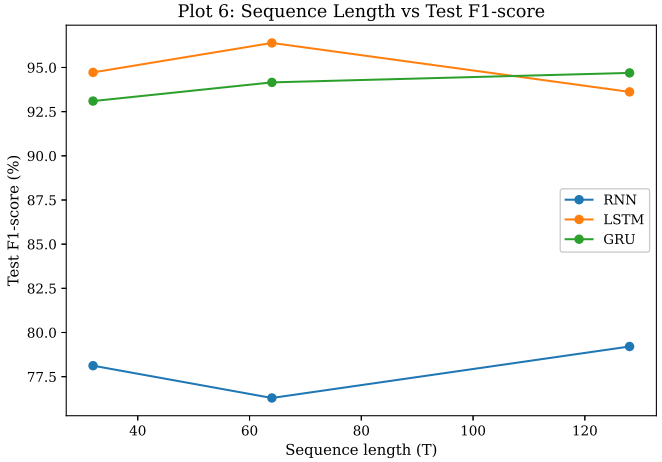
To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	78.12	94.72	93.10
64	76.29	96.39	94.16
128	79.21	93.62	94.69

Plot 6: Sequence Length vs. Test F1-score



Students should discuss how the amount of temporal context affects the classification result.

The effect of sequence length varies across the three models, showing that the amount of temporal context does not affect all architectures equally. For the RNN, the F1-score decreases slightly at sequence length 64 and then improves to 79.21% at 128, indicating that longer temporal context is beneficial. The LSTM performs best at sequence length 64, achieving the highest F1-score of 96.39%, but its performance decreases at 128, suggesting that excessive temporal context may not provide additional benefit. The GRU shows a gradual improvement as the sequence length increases, reaching its highest F1-score of 94.69% at 128. Overall, a moderate sequence length of 64 is optimal for LSTM, while longer sequences of 128 provide better results for RNN and GRU, demonstrating that the optimal temporal context depends on the model architecture.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

`https://www.crcv.ucf.edu/data/UCF101.php`

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D, then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

```
Predicted class : Basketball
Actual class    : Basketball
Confidence      : 0.xx
```

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

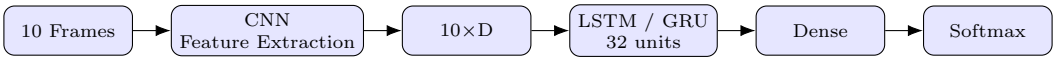
Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

With MobileNetV2 (`include_top=False, pooling="avg"`), the CNN feature dimension is $D = 1280$, so each 10-frame video clip is represented as a tensor of shape 10×1280, and a batch of B videos as $B \times 10 \times 1280$. (This is the fixed output width of MobileNetV2's global-average-pooled feature map; it did not depend on the unavailable UCF101 data and is reported here from the model definition in the notebook.)

21. CNN–LSTM / CNN–GRU Model

Use the following architecture:



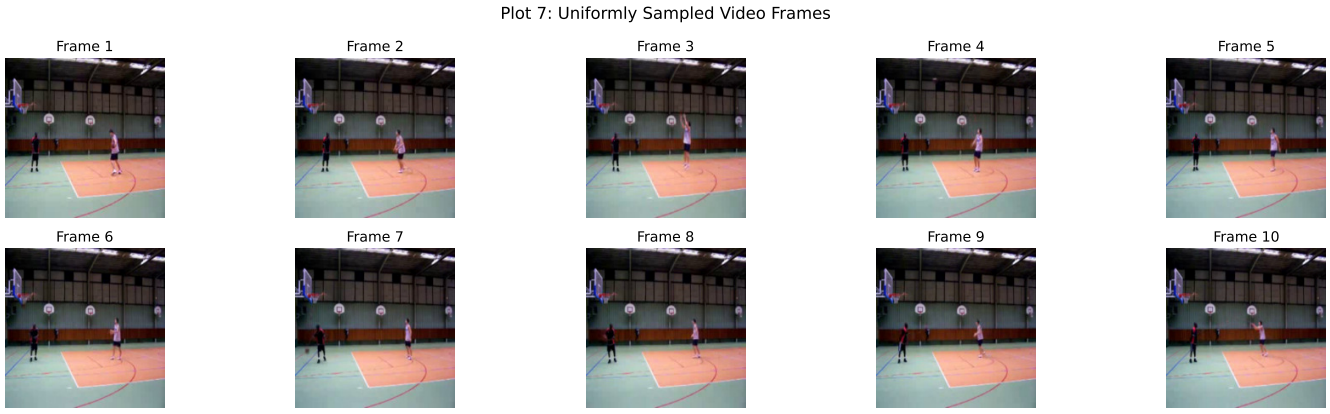
CNN Feature Extraction and Recurrent Input

The pretrained MobileNetV2 network was used as a frozen CNN feature extractor. Each video was represented using 10 uniformly sampled frames, with each frame resized to $224 \times 224 \times 3$. The CNN produced a feature vector of dimension $D = 1280$ for each frame.

Thus, the input to the recurrent network for one video has the shape $(10, 1280)$, while the complete batch has the shape $(40, 10, 1280)$. Therefore, the expected input format to the LSTM is $(B, 10, 1280)$, where B represents the batch size.

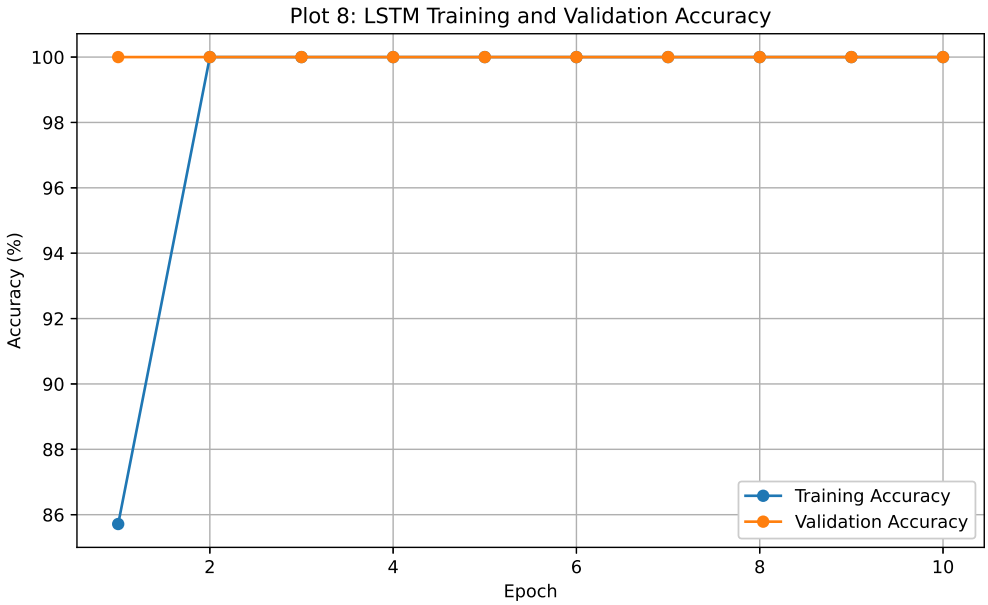
Property	Value
Frame shape	$(224, 224, 3)$
Number of frames per video	10
CNN feature dimension D	1280
Single video input	$(10, 1280)$
Batch input	$(40, 10, 1280)$

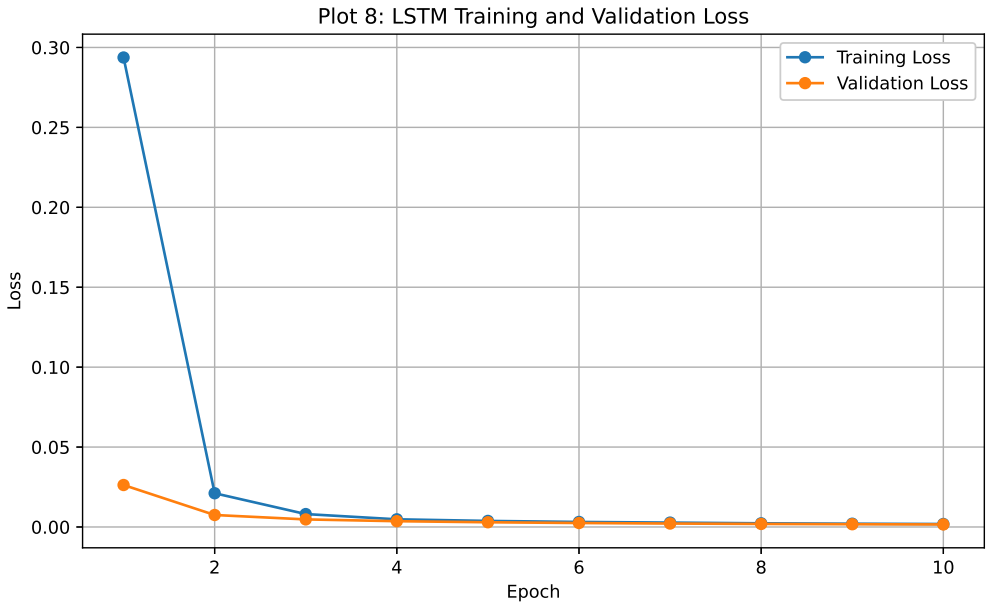
Plot 7: Video Sample Frames



Observation: The sampled frames show the visual content of the Basketball activity at different points in the video. The CNN extracts spatial information such as objects, body posture, scene appearance and visual patterns from each individual frame. The LSTM then processes these frame-level features in temporal order to learn how the visual information changes throughout the video.

Plot 8: Video Training and Validation Curves





Observation: The model converges rapidly, with training accuracy reaching 100% from the second epoch onward. Validation accuracy also remains at 100% throughout training, while both training and validation loss decrease steadily. The final training loss is 0.0019 and the final validation loss is 0.0016, indicating stable convergence and no noticeable generalization gap on the validation set.

Video Classification Performance

The CNN–LSTM model achieved a test accuracy of 100.00% with a test loss of 0.0018. Both selected action classes, *Basketball* and *Biking*, achieved perfect precision, recall and F1-score on the test set.

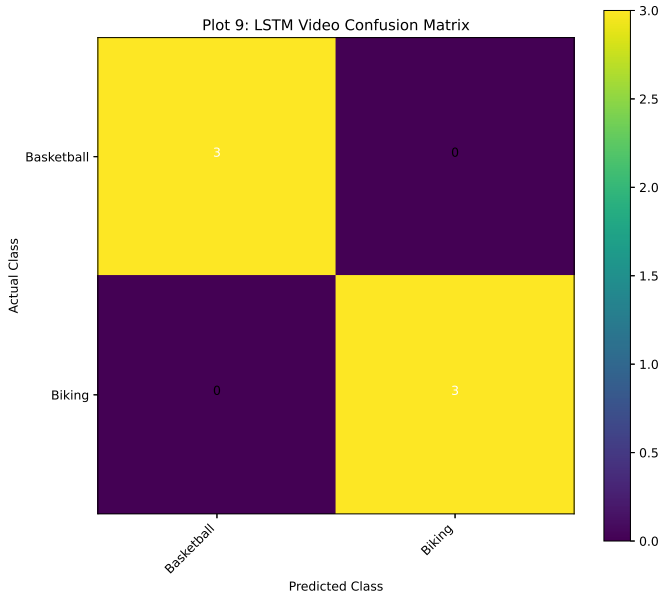
Class	Precision	Recall	F1-score	Support
Basketball	1.00	1.00	1.00	3
Biking	1.00	1.00	1.00	3

Video Prediction

For the representative test video, the model predicted *Basketball*, which matched the actual class. The confidence was obtained directly from the trained model's softmax output and was 99.90%.

Property	Result
Predicted class	Basketball
Actual class	Basketball
Model confidence	99.90%

Plot 9: Video Confusion Matrix



Observation: The confusion matrix shows that all three *Basketball* test videos and all three *Biking* test videos were classified correctly. There are no off-diagonal errors, indicating 100% class-level recognition for the two selected activities.

Overall Observation

The experiment demonstrates the effectiveness of combining CNN-based spatial feature extraction with recurrent temporal modelling. MobileNetV2 converts each 224×224 frame into a 1280-dimensional feature vector, while the LSTM processes the resulting sequence of 10 feature vectors to capture temporal information. The model achieved 100.00% test accuracy and a 99.90% model-generated confidence for the representative prediction.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder–decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

$$[5, 1, 8, 3].$$

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[0, 3, 9, 1]	[1, 9, 3, 0]
2	[6, 8, 6, 9]	[9, 6, 8, 6]
3	[2, 2, 6, 0]	[0, 6, 2, 2]
4	[5, 7, 7, 6]	[6, 7, 7, 5]
5	[2, 0, 7, 1]	[1, 7, 0, 2]

All five predicted outputs exactly match the expected (reversed) sequence.

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

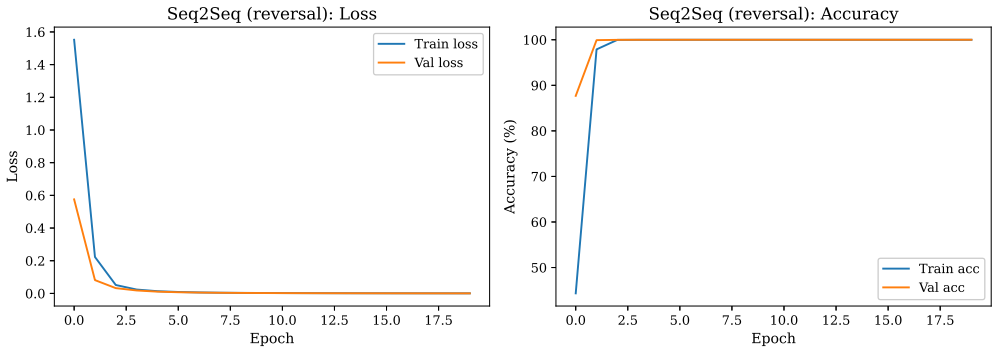
$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Results obtained (synthetic reversal task, $N = 20000$ sequences of length 4, digits 0–9, encoder–decoder LSTM with latent_dim $= 64$, embedding dim $= 16$, 20 epochs, batch size 64):

Metric	Value
Token accuracy	1.0000
Sequence accuracy	1.0000
Final training loss	0.0004
Final validation loss	0.0004



Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

Sequence-level accuracy can be substantially lower than token accuracy because a sequence is considered correct only when **all tokens in the sequence are predicted correctly**. Even a single incorrect token makes the entire sequence incorrect, whereas token accuracy counts each prediction independently. Therefore, small token-level errors can accumulate and significantly reduce sequence-level accuracy. In this case, both token and sequence accuracy are 1.0000, indicating that all tokens and complete sequences were predicted correctly. The very low training and validation losses of 0.0004 also indicate excellent model convergence and generalization.

25. Consolidated Results

Students must complete the following table.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Parameters
RNN	76.94	77.31	76.94	76.98	1974
LSTM	94.72	94.89	94.72	94.71	6006
GRU	95.83	95.85	95.83	95.83	4758

A separate table must be provided for the sequence-to-sequence task.

Task	Token Accuracy	Sequence Accuracy	Train Loss	Val Loss
Seq2Seq (reversal)	1.0000	1.0000	0.0004	0.0004

26. Discussion Questions

1. **What is a recurrent neural network?**
   An RNN is a neural network designed to process sequential data by maintaining a hidden state that carries information from previous time steps.
2. **What information is represented by the hidden state?**
   The hidden state stores a summary of relevant information from the current input and previous time steps.
3. **Why is sequence order important?**
   The order of elements determines their temporal relationships, which is essential for understanding patterns and dependencies in sequential data.
4. **Explain the difference between a feed-forward network and an RNN.**
   A feed-forward network processes each input independently, whereas an RNN maintains a hidden state to incorporate information from previous inputs.
5. **Explain Backpropagation Through Time.**
   BPTT trains an RNN by unrolling it across time steps and propagating the error backward through the entire sequence to update its weights.
6. **Why can gradients vanish during BPTT?**
   Repeated multiplication by small derivatives during backpropagation can make gradients progressively smaller, preventing earlier time steps from learning effectively.
7. **Why can gradients explode during BPTT?**
   Repeated multiplication by large derivatives can cause gradients to grow rapidly, resulting in unstable and excessively large weight updates.
8. **What are long-term dependencies?**
   Long-term dependencies are relationships where information from an earlier part of a sequence is required to correctly interpret a later part.
9. **What is the role of the LSTM cell state?**
   The LSTM cell state acts as a long-term memory pathway that carries important information across time steps while reducing the vanishing-gradient problem.
10. **What are the forget, input and output gates in LSTM?**
    The forget gate controls what information is removed, the input gate controls what new information is stored, and the output gate controls what information is exposed.
11. **What are the reset and update gates in GRU?**
    The reset gate controls how much previous information is ignored, while the update gate determines how much old information is retained versus replaced by new information.
12. **Compare LSTM and GRU.**
    LSTM uses a separate cell state and three gates, whereas GRU combines the memory and hidden state and uses two gates, making it simpler and more parameter-efficient.
13. **Why does GRU generally use a simpler gating mechanism than LSTM?**
    GRU combines related operations and does not maintain a separate cell state, resulting in fewer gates and fewer trainable parameters.
14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?**
    Using identical data, preprocessing, training conditions and evaluation metrics ensures that performance differences are due to the architectures rather than experimental factors.
15. **What does the confusion matrix reveal that accuracy alone does not?**
    A confusion matrix shows the individual class-wise predictions and identifies which classes are frequently confused with one another.
16. **Why should macro F1-score be reported for multi-class classification?**
    Macro F1 gives equal importance to every class by averaging their individual F1-scores, making it useful when class performance needs to be evaluated fairly.
17. **What is the significance of sequence length?**
    Sequence length determines how much temporal context the model receives and can affect both predictive performance and computational cost.

18. **Why can a CNN be used as a feature extractor for video?**
    A pretrained CNN can extract useful spatial features from individual video frames without requiring the CNN to be trained from scratch.
19. **What type of information is extracted by the CNN?**
    The CNN extracts spatial information such as shapes, textures, edges, objects and visual patterns present within each frame.
20. **What type of information is learned by the recurrent network?**
    The recurrent network learns temporal relationships and changes between consecutive frame features to recognize the overall activity.
21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?**
    Each frame produces a feature vector, and arranging these vectors in temporal order allows the recurrent network to learn how visual information changes over time.
22. **Explain the encoder and decoder in sequence-to-sequence learning.**
    The encoder converts the input sequence into a learned representation, while the decoder uses this representation to generate the required output sequence step by step.
23. **What is teacher forcing?**
    Teacher forcing trains a sequence-to-sequence decoder by providing the correct previous output as its input instead of using its own previous prediction.
24. **Differentiate token accuracy and sequence accuracy.**
    Token accuracy measures the percentage of individual tokens predicted correctly, whereas sequence accuracy considers a sequence correct only when all its tokens are correct.
25. **Why must the test set remain untouched during model selection?**
    Keeping the test set separate prevents information leakage and ensures that the final evaluation provides an unbiased estimate of the model's generalization performance.

27. Additional Exercises

1. **Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.**

Model	Units	Accuracy (%)	F1 (%)	Parameters	Time (s)
RNN	16	69.72	69.63	790	19.5
RNN	32	78.61	78.55	1974	17.8
RNN	64	84.17	84.23	5878	19.0
LSTM	16	85.83	86.03	2038	13.7
LSTM	32	95.00	94.98	6006	13.3
LSTM	64	94.44	94.40	20086	13.5
GRU	16	93.89	93.92	1670	13.1
GRU	32	94.44	94.37	4758	13.3
GRU	64	95.00	94.98	15542	13.5

   Observation: Increasing the number of units substantially improves RNN performance, with F1 increasing from 69.63% to 84.23%. LSTM performs best at 32 units, while GRU reaches its highest F1-score at 64 units. However, increasing the number of units also increases the parameter count considerably, while the training time changes only slightly.
2. **Compare GRU and LSTM using the same number of recurrent units.**

Units	LSTM F1	GRU F1	LSTM Params	GRU Params	LSTM Time (s)	GRU Time (s)
16	86.03	93.92	2038	1670	13.7	13.1
32	94.98	94.37	6006	4758	13.3	13.3
64	94.40	94.98	20086	15542	13.5	13.5

   Observation: GRU uses fewer parameters than LSTM for every unit configuration while achieving comparable F1-scores. GRU performs better at 16 and 64 units, whereas LSTM has a slight advantage at 32 units. Overall, GRU provides competitive performance with lower model complexity.
3. **Add a second recurrent layer and investigate whether performance improves.**

```
Stacked 2-layer LSTM: Acc=94.44%  F1=94.37%
Single-layer LSTM:    F1=94.71%
```

   Observation: Adding a second LSTM layer does not improve the performance in this experiment. The stacked model achieves an F1-score of 94.37%, slightly lower than the 94.71% obtained with a single layer. Thus, the additional recurrent layer increases complexity without providing a performance benefit for this dataset.
4. **Compare a bidirectional LSTM with a unidirectional LSTM.**

```
Unidirectional LSTM: F1=94.71%  Params=6006   Time=19.3s
Bidirectional LSTM : F1=95.82%  Params=11894  Time=22.5s
```

   Observation: The bidirectional LSTM achieves a higher F1-score of 95.82% compared with 94.71% for the unidirectional model. However, it uses nearly twice as many parameters and requires more training time. The improvement indicates that information from both temporal directions can be useful for this sequence classification task.

5. **Change the sequence length and study the effect on computational cost.**

Sequence Length	RNN Time (s)	LSTM Time (s)	GRU Time (s)	RNN Params	LSTM Params	GRU Params
32	14.5	11.7	10.9	1974	6006	4758
64	14.3	12.0	12.2	1974	6006	4758
128	18.4	13.4	13.2	1974	6006	4758

Observation: The number of trainable parameters remains unchanged when the sequence length increases because the same recurrent weights are shared across all time steps. Training time generally increases for longer sequences, with the largest increase observed at sequence length 128. This is because the models must perform recurrent computations over more time steps.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

$$\boxed{\text{Sequential Data} \rightarrow \text{Preprocessing} \rightarrow \text{RNN/LSTM/GRU} \rightarrow \text{Evaluation}}$$

and the video-understanding pipeline:

$$\boxed{\text{Video} \rightarrow \text{Frames} \rightarrow \text{CNN Features} \rightarrow \text{LSTM/GRU} \rightarrow \text{Action Prediction}}$$

They should also understand the encoder–decoder formulation:

$$\boxed{\text{Input Sequence} \rightarrow \text{Encoder} \rightarrow \text{Context} \rightarrow \text{Decoder} \rightarrow \text{Output Sequence}}$$

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student's own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
3. Kyunghyun Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014.
4. Anguita et al., "A Public Domain Dataset for Human Activity Recognition Using Smartphones," ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, "UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild," 2012.
6. TensorFlow Documentation: `https://www.tensorflow.org`
7. Keras Documentation: `https://keras.io`